Can LLMs be Effective Code Contributors? A Study on Open-source Projects

Chun Jie Chong
New Jersey Institute of Technology
Newark, New Jersey, USA
cc255@njit.edu

Muyeed Ahmed
New Jersey Institute of Technology
Newark, New Jersey, USA
ma234@njit.edu

Zhihao Zephyr Yao
New Jersey Institute of Technology
Newark, New Jersey, USA
zhihao.yao@njit.edu

Iulian Neamtiu
New Jersey Institute of Technology
Newark, New Jersey, USA
ineamtiu@njit.edu

Abstract

LLM-generated code is widely used, and the share of committed code produced by LLMs is expected to increase. However, we are not at a point where LLMs can be effective contributors to production code. We present an approach that exposes the shortcomings of LLM generation on such projects, and proposes recommendations; the targets of our study are sizable open-source projects, e.g., FFmpeg and wolfSSL. First, we developed a framework that uses verification and validation to evaluate a given LLM's suitability to fix or add features to an existing project. Second, we apply the framework to 212 commits (bug fixes and small feature improvements) in eight popular open-source projects and three LLMs: GPT-4o, Ministral3, and Qwen3-Coder. The success rate varied from 0% to 60% depending on the project. The LLMs failed in a variety of ways, from generating syntactically incorrect code, to producing code that fails basic (static) verification, or validation via the project's test suite. In particular, the LLMs struggle with generating new code, handling contexts (function or file) outside a certain size range, and in many cases their success is due to parroting code changes they have been trained on.

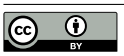
CCS Concepts

• **Software and its engineering → Software verification and validation**.

Keywords

Empirical Study, Large Language Models, Code Generation, Code Quality

ACM Reference Format:
Chun Jie Chong, Muyeed Ahmed, Zhihao Zephyr Yao, and Iulian Neamtiu. 2026. Can LLMs be Effective Code Contributors? A Study on Open-source Projects. In *International Conference on Evaluation and Assessment in Software Engineering (EASE '26), June 09–12, 2026, Glasgow, United Kingdom.* ACM, New York, NY, USA, 11 pages. https://doi.org/10.1145/3816483.3816490



This work is licensed under a Creative Commons Attribution 4.0 International License.
EASE '26, Glasgow, United Kingdom
© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2348-3/26/06
https://doi.org/10.1145/3816483.3816490

1 Introduction

Thanks to Large Language Models (LLMs) advancements, AI-assisted programming is increasingly adopted in software development [6]. AI code editors such as GitHub Copilot [4] (powered by LLMs including OpenAI's GPT [5]), Visual Studio IntelliCode [11], or Cursor [9], help programmers with code suggestions, explanations and implementation. Accelerating software development is one of the reported benefits, e.g., a study found that 88% of developers reported an increase in productivity when using Copilot [2].

However, LLM use comes with trade-offs. Studies have shown that LLMs misuse APIs [30], create irrelevant and complicated conditional logic [30], lack defensive programming [22], introduce security issues [22], and hallucinate [31]. Iterative prompting might fix some of these issues, but the results from iterative prompting are not consistent and not guaranteed [22, 40]. While benchmarks have been introduced to evaluate LLM-generated code [20, 34, 49] they might not be representative of commits "in the wild" and LLMs could overfit to perform well on benchmarks alone. These benchmarks also fail to capture the whole scope of real-world software evolution, e.g., code integration and testing (unit or whole-system).

We believe that an effective LLM must provide a fully automated solution that, *given a sizable code base and a prompt, generates a high-quality, usable commit* (patch) that satisfies the prompt specification and can be directly incorporated into the base. Towards that goal, several issues must be addressed: (1) how to assess whether LLMs are suitable at generating commit-ready code, and (2) when LLMs fail, how do they fail, and what can we do about it.

To tackle these issues and help make LLMs more effective at contributing to sizable code bases, we developed an automated framework that evaluates LLMs' ability to fix bugs and implement new features via static analysis and test suites. Further comparisons between human-written and LLM-generated code allow us to identify LLM-generated code issues that escape the purview of automated verification&validation. Our automated framework uses programming tasks (commits) from open-source repositories, which are more representative than generic benchmarks. For our study, we focus on 212 actual commits (187 bug fixes and 25 feature enhancements) in 8 open-source projects: Bison, Collections-C, FFmpeg, jansson, libhl, packcc, Vsftpd, and wolfSSL. We focus on C code as it is widely spread in system programming and open-source projects.

Bug

```
const size_t c = deque->capacity - 1;
const size_t l = deque->last & c;
const size_t f = deque->first & c;
const size_t p = (deque->first + index) & c;
void *removed  = deque->buffer[index];

if (out)
    *out = removed;
```

Should be replaced by p

Human fix

```
const size_t c = deque->capacity - 1;
const size_t l = deque->last & c;
const size_t f = deque->first & c;
const size_t p = (deque->first + index) & c;
void *removed  = deque->buffer[p];

if (out)
    *out = removed;
```

GPT-4o fix

```

  Removed 3 variables: c, l and f
size_t p = (deque->head + index) % deque->capacity;

if (out) {
    *out = deque->buffer[p];
}
```

Modified the calculation of p

Figure 1: Human vs. GPT-4o fixes for an incorrect index bug.

Bug

```
if(ours)
    jsonp_free(key);
set_error(s, "<internal>", "Unable to add key \"%s\"
    ", key);
goto error;
```

Using *key* after it has been freed

Human fix

```
set_error(s, "<internal>", "Unable to add key \"%s\"
    ", key);

if(ours)
    jsonp_free(key);
goto error;
```

Call *set_error* before freeing *key*

GPT-4o fix

```
if (ours)
    jsonp_free(key);
json_decref(value);
set_error(s, "<internal>", "Unable to add key \"%s\"
    ", key);
goto error;
```

Invalid, *value* does not have ref count

Did not fix the use-after-free error

Ministral3 fix

```
if (ours)
    jsonp_free(key);
json_decref(object);
set_error(s, "<internal>", "Unable to add key \"%s\"
    ", key);
return null;
```

Did not fix the use-after-free error

Qwen3-Coder fix

```
if (ours)
    jsonp_free(key);

set_error(s, "<internal>", "Unable to add key \"%s\"
    ", key);
goto error;
```

No changes applied

Figure 2: LLM-generated code for fixing a use-after-free bug.

Section 2 presents several motivational examples – four actual tasks from commits – along with the issues in the LLM-generated code. Section 3 describes our approach: the framework, the projects and commits used for evaluation, and the verification&validation processes. Section 4 presents our study results along several Research Questions (RQs) designed to check LLMs' ability to generate effective code contributions, i.e., Can LLMs generate code that compiles, passes static analysis, and passes testing? When it fails, why and how does it fail? Which factors affect success rate?

We applied our framework to three LLMs: GPT-4o, Ministral3-14B, and Qwen3-Coder-30B. We now summarize some of our findings. Though the 212 commits were small, typically 4–15 LOC, the LLM-generated code fails to compile in 2–32 cases (depending on the LLM). Even when the code compiles, the verification stage found 9–18 null pointer dereferences, and 9–72 unsafe casts. Among those 98–143 commits on which we could run a test suite, the LLMs had a passing rate of 71.8–86.7%. The main causes for validation failure included partial fixes, empty generated patches, fixing the wrong thing, or deleting unrelated code. We found that context size (subject file or target function) influences the success rate, e.g., a lower rate for larger contexts. In Section 5 we provide recommendations on how to increase effectiveness and safety when using LLM generation.

Open Science policy. Our artifact is available at https://github.com/redoubt-lab/LLM-Evaluation-Framework.

2 Motivational Examples

The fundamental question we ask is "Why can't LLMs be effective contributors on sizable projects"; once we know how they fail in our setting – a variety of tasks from a variety of files in a variety of projects – we can generalize to larger contexts, and proceed to address or mitigate some of the issues. We motivate our approach by showing the generated code if we asked LLMs to produce the code required for typical maintenance and evolution needs. The four tasks are actual commits (bug fixes and feature enhancements) from our examined projects; the LLMs were given the affected file as context and the actual commit message as prompt. Due to space constraints, we show the code output from all three LLMs for only two tasks, and one LLM for the other two tasks. The first commit allows us to check whether an LLM can complete a straightforward task, as the fix is suggested in the prompt and requires changing a single line of code. The second and third commits involve fixing use-after-free and null pointer dereference errors, which are among the 2024 CWE Top 25 Most Dangerous Software Weaknesses list [3]. LLMs' ability to solve these tasks is critical, as these vulnerabilities are common in software projects. The last commit is a feature enhancement that requires understanding of the data structure used; this task examines LLM's understanding of the context. For these tasks, we use the commit message as part of the prompt to the LLM and attach the affected file as contextual reference. The prompt we used for fixing a bug was "*Modify the function in the provided C file such that it fixes the following issue: <**commit message**> in <**function name**>*"; for implementing a new feature, we used "*Implement a function in the provided C file: <**function name**> such that it <**commit message**>*".

Bug

```
while (replace) {
    if (table->key_cmp(replace->key, key) == 0) {
        replace->value = val;
        return CC_OK;
    }
}
```

Could be NULL

Human fix

```
while (replace) {
    void *rk = replace->key;

    if (rk && table->key_cmp(rk, key) == 0) {
        replace->value = val;
        return CC_OK;
    }
}
```

NULL check added

GPT-4o fix

```
 while (replace) {

    if (table->key_cmp(replace->key, key) == 0) {
        replace->value = val;
        return CC_OK;
    }
}
```

No changes applied

Figure 3: GPT-4o-generated code for fixing a null pointer dereference error.

2.1 Fixing an Incorrect Index Bug

This commit, from *Collections-C* [8], fixes a function that returns an element at the wrong index.[1] Figure 1 shows the bug, the human fix, and the GPT-4o-generated fix. The bug is shown on line 6; the human fix correctly changed `deque->buffer[index]` to `deque->buffer[p]`. The commit message is: *"The* `deque_remove_at` *function was returning an incorrect element in certain situations. The correct element is the one at index* `p`*"*. However, GPT-4o fails to fix the bug; on the right we highlight the three issues with the GPT-4o-generated code. First, the calculation of `p` is modified, which leads to accessing the element at the wrong index again. Second, GPT-4o removes certain variables that are irrelevant to the fix. Third, GPT-4o uses an undeclared variable, `deque->head`, which causes a compilation error, despite the fact that the entire file (prior to the commit) was provided as context, including the declaration of the deque struct.

2.2 Fixing a Use-after-free Error

This commit, from *jansson* [12] involves fixing a potential use-after-free error when generating an error message.[2] The code, shown in Figure 2, has a path on which a pointer `key` is used as an argument to `set_error` after it has been freed via `jsonp_free(key)`. The fix involves moving the call to `set_error` before invoking the `jsonp_free` function on it, as shown in the human code (center-left). The commit message used for the prompt was: *"Fix a use after free"*. However, all three LLMs fail to identify this problematic issue. In the GPT-4o-generated code, the variable `key` is still being used after it has been freed. Furthermore, GPT-4o attempts to fix the use-after-free error by decrementing the reference count of `value` at line 3, which reflects a misuse of the function, since `value` is not an allocated object in this context and does not have a reference count. Ministral3 replaces the line `goto error` with `return null` and a call to decrement the reference count of `object` at line 3, which does not change the program as the `error` label performs the same operations; this change does not fix the use-after-free error. Qwen3-Coder makes no changes, and hence does not fix the error.

2.3 Fixing a Null Pointer Dereference

This commit, from *Collections-C* [8], fixes a potential null dereference (which could lead to undefined behavior such as a segmentation fault).[3] The buggy code, shown in Figure 3 (top), invokes `strcmp` (`table->key_cmp` uses `strcmp` under the hood) on `replace->key`, which could be `NULL`. The human fix consisted of adding a `NULL` check on `replace->key` to prevent the null pointer dereference. The commit message used for the prompt was: *"Fixes a bug that would be triggered by add, get and remove hashtable operations when a key whose hash value would resolve to table index 0 was used while NULL key was already present in the table"*. Figure 3 demonstrates one of the three functions that GPT-4o was asked to fix. GPT-4o did not make any changes to the three functions, even though it confidently stated that the generated code has fixed the issue.

2.4 Implementing a Function for Size Retrieval

This commit, from *jansson* [12] implements a new function that returns the hash table size in a `struct` called `json_object_t`.[4] Figure 4 shows the function, which first checks whether the parameter is an object, then casts it to `json_object_t` if the check passes, and returns `object->hashtable.size`. The commit message only mentions the name of the added function. Therefore, for prompt construction, we use the function description, stated in the human commit: *"Returns the number of elements in *object*, or 0 if *object* is not a JSON object"*. The first difference is that the human implementation deviates from the function description by returning `-1` instead of `0`, whereas all three LLMs follow the prompt and return `0`. The second difference is that GPT-4o's and Ministral3's solutions use an undeclared function, `hashtable_size`, whereas Qwen3-Coder's solution uses an undeclared `struct` member. The function and `struct` are not implemented by the LLMs, but rather the LLMs assume that they exist in the codebase. These cause compilation errors, as expected.

3 Approach

Figure 5 shows an overview of our approach: a general *framework*, that evaluates a given LLM on a given repository, with a given prompt, and an *evaluation* that applies this framework to existing commits from our examined projects.

Framework. The framework is designed to verify and validate LLM-generated code, which can be obtained either (1) by providing a custom prompt or (2) from a commit in a GitHub repository. The framework takes a source file and a prompt as input which are

[1] https://github.com/srdja/Collections-C/commit/5ba7eed
[2] https://github.com/akheron/jansson/commit/11813f4
[3] https://github.com/srdja/Collections-C/commit/2843fe2
[4] https://github.com/akheron/jansson/commit/1e00cd5

Human

```c
unsigned int json_object_size(const json_t *json)
{
    json_object_t *object;
    if(!json_is_object(json))
        return -1;

    object = json_to_object(json);
    return object->hashtable.size;
}
```

Implementation differs from the specification

Returns -1 instead of 0

GPT-4o

```c
unsigned int json_object_size(const json_t *json )
{
    if (!json_is_object(json))
        return 0;

    json_object_t *object = json_to_object(json);


    return hashtable_size(&object->hashtable);
}
```

hashtable_size() is undeclared

Ministral3

```c
unsigned int json_object_size(const json_t *json)
{
    if(!json_is_object(json))
        return 0;
    json_object_t *object =
        json_to_object(json);

    return hashtable_size(&object->hashtable);
}
```

hashtable_size() is undeclared

Qwen3-Coder

```c
unsigned int json_object_size(const json_t *json )
{
    if (!json_is_object(json))
        return 0;

    const json_object_t *object =
        json_to_object(json);


    return object->hashtable.entries;
}
```

hashtable.entries is undeclared

Figure 4: LLM-generated code for implementing a size retrieval function.

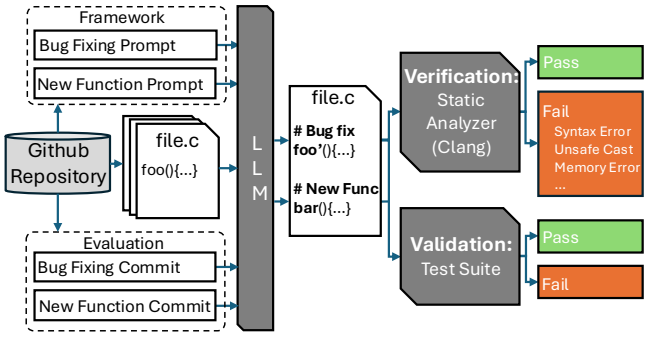


Figure 5: Overview of our approach.

passed to an LLM to generate a candidate solution. This solution is integrated in the original source code and then verified with the Clang static analyzer [1] (e.g., for safety issues), followed by validation via the project's test suite (if available).

Evaluation. We applied our framework to the actual commits from our examined projects. We automated prompt generation by leveraging the actual commit message and the function name affected by the commit. Specifically, given a repository and a commit that describes a bug fix or new function implementation, we extract the relevant function (e.g., *foo()*, *bar()*) from the commit diff and construct a prompt using one of two templates shown in Section 2, depending on the nature of the change. We also identify the changed file (e.g., `file.c`) and retrieve its previous version from the commit's parent, which we then attach as input to the LLM. The generated function, either a replacement (*foo'()*) or a new implementation (*bar()*) is inserted into `file.c`. Essentially this yields a code base where the only difference is the LLM-generated patch instead of the original human patch; we run verification and validation on this code base.

No loop. We deliberately eschewed iterative prompting, i.e., we did not re-prompt in case the LLM-generated code was unsatisfactory. We made this choice for two reasons. First, LLMs should save developers time, not trade development time for iterative prompting time. Second, recent research has shown that iterative prompting might degrade the quality of the output or even introduce bugs in bug-free code [22].

3.1 LLMs Used for Evaluation

For our study, we used three LLMs: one in the cloud and two local LLMs. First, we used OpenAI's GPT-4o, due to its API availability and widespread adoption. Second, we used Ministral3 with 14 billion parameters and Qwen3-Coder with 30 billion parameters as local LLMs. As smaller LLMs are closing the gap with larger LLMs, we wanted to see how well these smaller LLMs perform as compared to larger LLMs such as GPT-4o. In addition, local LLMs are more suitable for the development process, as they incur no token or API fees, work offline, and ensure data privacy. Using a different LLM involves simply changing the API endpoint.

3.2 Projects Used for Evaluation

To bolster external validity [25], i.e., choose representative GitHub projects, we proceeded in line with previous research [26, 37]. Specifically, we selected projects based on four criteria: project size, context size, context size range, and popularity. We defined a minimum project size, 1500 LOC, and a median file size of at least 400 LOC to evaluate LLM performance on realistic setups, and projects likely to come with tests [29]. We chose projects where context size range (max/min) was at least 7x, to cover a wide range of file context sizes. Finally, we only considered projects above a certain popularity, measured by the number of stars (>300) and forks (>30), which indicates a diverse contributor list.

We examined eight open-source projects that met these criteria: six hosted on GitHub, and two hosted elsewhere. Table 1 shows

Table 1: Summary of project, commit, and context properties.

Name	Stars	Forks	Commit Timespan (MM/YY)		Project Size (LOC)		Context (File) Size (LOC)			#Commits	
			Start	End	Start	End	Min	Median	Max	Feature Enhancements	Bug Fixes
Collections-C	2,900	330	09/14	05/25	2,006	6,746	81	493	942	3	17
packcc	371	33	07/20	05/24	3,833	5,443	3,833	4,465	5,442	0	5
libhl	442	119	10/13	06/14	1,750	5,849	103	557	741	1	19
jansson	3,200	833	09/09	02/15	1,882	4,142	32	667	779	11	9
FFmpeg	51,200	12,800	01/12	06/25	473,858	1,340,782	65	1,066	10,319	2	73
wolfSSL	2,500	876	12/23	12/24	1,345,510	1,495,448	253	23,923	125,527	0	22
Vsftpd	n/a	n/a	08/09	08/21	15,302	16,415	37	656	2,554	4	22
Bison	n/a	n/a	08/18	09/21	19,681	25,480	98	430	824	4	20

projects' characteristics. The GitHub projects were: (1) Collections-C [8] – a data structure library; (2) packcc [14] – a parser generator; (3) libhl [13] – a thread-safe data structure library; (4) jansson [12] – a library to process JSON data; (5) FFmpeg [10] – a framework for processing audio and video; (6) wolfSSL [17] – a lightweight SSL/TLS library. We also included two open-source projects, the Bison parser generator [7] and the Vsftpd FTP server [16]; while not hosted on GitHub, their release histories are available, as changelog entries, similar to GitHub commit messages. Moreover, Bison and Vsftpd are the default FTP server and parser, respectively, in popular Linux distributions (Debian/Red Hat and derivatives), which confirms their popularity. All projects were written mainly in C.

3.3 Commit Selection Process

We selected 212 commits, all targeting the C code in the projects: for GitHub projects we used actual commits, while for Bison and Vsftpd, we selected the tasks from changelog entries (for uniformity, we refer to these entries as commits as well). The number of commits, and their kind, for each project, are shown in the last two columns of Table 1. The commits were selected based on the following criteria.

Single file and single function change. This criterion keeps the tasks simpler for LLMs. The rationale behind it is that LLMs are known to struggle on complex tasks, e.g., LLMs are more likely to solve tasks that only take humans 11 minutes or less [50].

Manageable commit size. The selected commits' median sizes were 4 LOC for bug fixes and 15 LOC for feature enhancement (Table 2). When a solution exceeds 12 LOC, 60% of LLM-generated code tends to be irrelevant to the actual fix [45]; in such cases chain-of-thought prompting [46] could be used. This prompting technique is recommended by GitHub [15] and has been shown to improve LLMs' ability to perform complex tasks. However, due to the nature of feature enhancement commits – requiring the implementation of a new function – their size is slightly higher than for bug fixes.

Well-formed commit messages and changelog entries. A well-formed commit message contains two main parts: (a) what has changed, and (b) the rationale for the change [19, 35]. A study shows that typically 56% of open-source projects' commits contain the "why" and "what" [42].

Bug fix variety. The commits were selected to cover: (1) functional, project-specific issues, and (2) a range of common errors such as null pointer or indexing errors. We believe LLMs should be capable of fixing both common C issues and project-specific bugs.

Table 2: LOC changes in the selected commits.

Commit	Min	Max	Median	Mean
Bug Fix	1	48	4	6.61
Feature Enhancement	6	66	15	19.96

Feature enhancements. These involved implementing short, simple, new functions (human equivalent: ≈15 LOC) that might call existing functions from the provided context file.

Project lifespan. We looked for projects with a long lifespan, to ensure the availability of sufficient commits with a long development history, and capture a wide range of growth phases in the projects' evolution. While project lifespan is omitted from Table 1 for space reasons, our examined projects' lifespans were at least 5 years (in some cases exceeding 10 years).

Commit dates span pre- and post-cutoff dates. We performed a focused study (GPT-4o only) on the effect of LLM's knowledge cut-off date. The first period was before GPT-4o's cutoff date (May 2024). Since these pre-cutoff commits are likely to be in the training data, we expect GPT-4o to do well and cite sources. The second period comes after the training cutoff date, which evaluates GPT-4o's performance on unseen data. Ministral3 and Qwen3-Coder do not specify knowledge cutoff dates; the models themselves were released in December 2025 and July 2025, respectively.

3.4 Static Analysis

For verification we use the Clang static analyzer [1], to keep the process scalable for the project size and number of commits we cover. Clang offers a set of default checkers that checks for issues, such as division by zero, dereference of NULL pointers, and use of uninitialized variables. We also enable experimental checkers in Clang to check for a wider range of issues, such as out-of-bounds access in string functions and invalid casting between pointers.

3.5 Test Suites

For validation, we use the projects' test suites. Test suites are run to achieve the following two objectives: (1) unit testing, ensuring the LLM-generated code fixes the bug and implements the intended functionality; and (2) regression testing, making sure the LLM-generated code does not break existing functionality.

Table 3: Categories of issues from static analysis. H: human; G: GPT-4o, M: Ministral3, Q: Qwen3-Coder (LLMs).

Category	#Issues in Code			
	H	G	M	Q
Null Dereference	23	18	16	9
Undeclared Identifier	0	31	50	20
Unsafe Cast	34	64	72	9
Double Free	0	1	0	2
Use After Free	0	1	1	1
Syntax/Semantic	0	40	45	1
Uninitialized Variable	4	4	0	0
Empty/Partial	0	0	7	44
Total	*61*	*159*	*195*	*97*

3.6 Manual Inspection

We also manually inspected the LLM-generated code to confirm it implements the intended feature or fix. This was necessary as we observed that LLM-generated code could pass all the tests even though the correct fix or feature was not implemented, specifically in commits that did not add test cases checking the fix or feature. The generated commits were evaluated independently by two PhD students. We confirmed a 100% inter-rater agreement rate between the patch assessments.

4 Findings

We now discuss the findings of our evaluation, structured around four research questions. Note that due to computational limitations, we were unable to generate code for 14 out of 22 commits from wolfSSL using Qwen3-Coder when the context file size exceeded 13,000 LOC. Since the context file was included as part of the prompt for local LLMs, a large context file substantially increases the token count, which requires more memory to process the prompt. We used a MacBook Pro with 36GB RAM for local LLM code generation.

4.1 RQ1: Can LLMs Generate Syntactically Correct Code?

Failure to compile is one of the prominent issues observed in LLM-generated code. This is due to the frequent use of undeclared identifiers, such as functions, variables, and `struct` members. Undeclared identifiers account for 31 (19.5%), 50 (25.6%), and 20 (20.6%) issues for GPT-4o, Ministral3, and Qwen3-Coder respectively, as shown in Table 3. For instance, Figure 4 illustrates the LLMs' assumptions that `hashtable_size` and `hashtable.entries` exist in the codebase, despite their absences in the provided context files. Another noticeable category is syntax and semantic errors; we observed 40 (25.2%), 45 (23.1%), and 1 (1%) such cases across the three LLMs. For example, the use of `case` statement outside `switch` statement, missing curly braces, and treating a variable of type `uint8_t` as a `struct`, are grouped as syntax and semantic issues. These issues cause compilation errors, as expected (the human-written code does not have such issues, because it would not be accepted as a commit). Though such issues are exposed at compile time, fixing them will require a time-consuming human intervention, which undermines the goal of accelerating software development using LLMs. Note that Qwen3-Coder has an ostensibly lower rate for syntax and semantic errors because it generated empty or partial code for 44 commits, whereas Ministral3 has 7 such cases; uncompilable code vacuously leads to fewer issues during static analysis.

Table 4: Verification (static analysis) results.

Projects	#Issues in Code				#Empty/Partial Code		
	H	G	M	Q	G	M	Q
packcc	0	0	1	0	0	0	0
jansson	0	1	17	5	0	0	0
Collections-C	0	8	0	3	0	3	3
libhl	3	4	22	5	0	0	7
FFmpeg	56	128	99	20	0	0	26
wolfSSL	0	5	20	0	0	2	7
Bison	1	12	18	10	0	0	1
Vsftpd	1	1	11	10	0	2	0
Total	*61*	*159*	*188*	*53*	*0*	*7*	*44*

Table 5: Test suites results on LLM-generated code. # Tests means the total number of tests available.

Projects	Pass			Fail			Compile error			#Tests
	G	M	Q	G	M	Q	G	M	Q	
packcc	2	1	3	1	2	0	2	2	2	5
jansson	15	12	15	4	4	4	1	4	1	20
Collections-C	16	15	13	2	2	1	2	3	6	20
libhl	13	10	10	6	7	2	0	2	7	19
FFmpeg	56	34	27	0	6	3	0	16	26	56
wolfSSL	17	7	2	0	0	0	0	10	15	17
Bison	5	5	7	6	12	11	9	3	2	20
Total	*124*	*84*	*77*	*19*	*33*	*21*	*14*	*40*	*59*	*157*

4.2 RQ2: Why Does Code Fail Static Verification?

We performed static analysis on both human-written and LLM-generated code for the selected commits. Table 4 shows that GPT-4o-generated code has a total of 159 issues (160% more than human-written code), whereas Ministral3-generated code has a total of 188 issues (208% more). Qwen3-Coder has a total of 53 issues (13.1% less than human-written code); however, 44 of its commits are either empty or contain only partial code. The numerous issues found in LLM-generated code are presented in Table 3 and discussed next.

NULL Pointer Dereferences. Null pointer dereferences are severe issues that can cause denial of service, which is especially a concern if the error happens in low-level systems software. For instance, as shown in Figure 3, GPT-4o's failure to fix the issue could lead to a program crash (segmentation fault).

Unsafe Casts. This category has the highest proportion for both GPT-4o (40.3%) and Ministral3 (36.9%), and accounts for 9.3% for Qwen3-Coder. Unsafe casts introduce a range of potential vulnerabilities: data loss, crashes and invalid reads or writes to unintended memory regions. For instance, the LLM-generated code `*dst++ = replacements[i]`, which attempts to assign a `char *` (`replacements[i]`) to a `char` (`dst`) causes an unsafe, pointer-to-character cast.

Table 6: Manual inspection results for LLM-generated patches. 'Success Rate'=fraction of patches judged correct by the inspection.

		packcc			jansson			Collec.-C			libhl			FFmpeg			wolfSSL			Bison			Vsftpd		
		G	M	Q	G	M	Q	G	M	Q	G	M	Q	G	M	Q	G	M	Q	G	M	Q	G	M	Q
Correct	Identical to Human Solution	0	0	0	3	2	5	6	8	9	6	4	4	3	1	5	1	0	0	0	0	0	1	2	3
	Different from Human, Appears Correct	2	0	2	9	4	3	6	2	1	5	2	3	7	3	4	2	0	0	10	4	7	8	2	2
Incorrect	Partial Fix	0	0	0	0	0	0	0	0	0	1	2	1	12	4	7	0	0	0	1	1	1	0	0	0
	LLM Did Nothing (Empty Patch)	0	0	0	1	0	1	1	5	5	0	1	9	8	12	21	2	6	7	1	0	1	2	2	3
	Wrong Solution, Failed to Fix or Implement	2	5	3	5	10	10	4	5	1	7	8	3	25	48	35	16	2	1	6	15	12	11	12	11
	Deleted Code Unrelated to the Task	1	0	0	1	0	0	0	0	1	1	2	0	2	3	3	0	0	0	1	0	1	3	1	0
	Uncompilable Code (Undeclared Identifiers)	0	0	0	1	4	1	3	0	3	0	1	0	18	4	0	1	1	0	5	4	2	1	7	7
Success Rate (GPT-4o)		*40%*			*60%*			*60%*			*55%*			*13.3%*			*15%*			*41.8%*			*34.6%*		
Success Rate (Ministral3)		*0%*			*30%*			*50%*			*30%*			*5.3%*			*0%*			*16.7%*			*15.4%*		
Success Rate (Qwen3-Coder)		*40%*			*40%*			*50%*			*35%*			*12%*			*0%*			*29.2%*			*19.2%*		

Free-Related Vulnerabilities. We observed two classes of free-related vulnerabilities: double free and use-after-free. Double free occurs when an allocated region is freed twice, while use-after-free involves accessing a variable after its memory has been deallocated. Both vulnerabilities can lead to undefined behavior.

Uninitialized Variables. Variables that are declared but not assigned to (prior to use) have undefined behavior, with various manifestations across compilers or machines, e.g., as program crashes.

4.3 RQ3: Validation: Why is Code Incorrect Even though It Passes Verification?

In addition to the static analysis-based verification, we also performed validation: we ran the projects' test suites, and followed that with a manual inspection.

Test Suites. We first ran the test suites on the human-written code for the selected commits; this code passed all test cases, as expected. Next, we replaced the human-written commit with the LLM-generated commit, and re-ran the test suite. We applied this procedure to all projects except Vsftpd, which lacks a test suite. For libhl, we were only able to run 19 test suites out of 20 commits, as one of the libhl versions does not contain a test suite. Note that we were also unable to run test suites on LLM-generated code that caused compilation errors, such as those resulting from the use of undeclared identifiers (Section 4.1). Therefore, the total number of test suites does not correspond to the total number of selected commits. In all, for 7 projects, we have a total of 157 test suites available, and we run 98, 117, and 143 test suites (across 98–143 commits) on Qwen3-Coder, Ministral3, and GPT-4o, respectively.

Table 5 shows test suite run results. GPT-4o-generated code has the highest number of passing test suites (124), followed by Ministral3 (84), and Qwen3-Coder (77). For failing test suites, Ministral3 has the highest number (33), followed by Qwen3-Coder (21), and GPT-4o (19). The reasons for failing test suites include not passing all the test cases and causing the test suite to deadlock and hang. We observed that *a high test suite passing rate does not guarantee code correctness.* Fixes or new features often do not come with new associated test cases in the commits. Therefore, LLMs' failure in fixing bugs or implementing new features may go unnoticed as the relevant test cases are missing from the test suites. Note that for Qwen3-Coder, there are 59 commits (37.6%) that cannot be compiled or used to run the test suites, while there are 40 (25.5%) such commits for Ministral3 and 14 commits (8.9%) for GPT-4o.

Manual Inspection. We manually inspected all 212 commits by comparing LLM-generated with human-written code and tracing the LLM-generated code with inputs that trigger the bugs (for bug-fixing commits) and inputs that served as use cases (for feature enhancement commits). Table 6 shows the results. We group incorrect LLM-generated code into 5 categories (category *Uncompilable Code due to Undeclared Identifiers* was discussed in Section 4.1).

Partial Fix. In such cases, the fix requires addressing an issue in, say, N locations in the given function, but the LLM addressed fewer than N. For instance, a commit from *libhl* [13] fixes a memory leak by releasing references of `next` and `prev` [5]. The bug is shown on lines 2 and 8 in Figure 6 (top). The human fix involves releasing references of `next` and `prev` on lines 2 and 8, respectively. The commit message is as follows: *"fixed a memory leak triggered by a fault path. the node was overretained"*. However, GPT-4o only released the reference on line 2, i.e., a partial fix.

LLM Did Nothing (Empty Patch). In such cases, the LLM did not make any changes to the given code, though it was asked to, in the prompt. Figure 3 illustrates this troublesome behavior: GPT-4o outputs code identical to the code prior to the commit, even though GPT-4o is confident in the response that it fixed the bug.

Wrong Solution, Failed to Fix or Implement. In such cases, the LLM attempted to fix a bug or implement a new feature, but the solution is incorrect; for example, GPT-4o and Ministral3 failed to address an issue in a single location within the function (Figure 2). This differs from a partial fix, where the LLM performed one or more fixes, but more changes are needed to fully resolve the issue.

Deleted Code that was Unrelated to the Task. We also observed that the LLM can delete code unrelated to the task, which in turn breaks existing functionality. For example, a commit from *packcc* [14] adds a feature to support escaped backslash.[6] The commit message used is *"Support escaped backslash for both string matching and character class matching"*. However, GPT-4o deleted 3 switch cases – `u`, `\n`, and `\r` – which were unrelated to the task, as shown in Figure 7. This caused the test suites to fail, as expected.

[5] https://github.com/xant/libhl/commit/af96d58
[6] https://github.com/arithy/packcc/commit/5211290

Bug

```
if (!prev) {

    destroy_entry(entry);
    return -1;
}
while (1) {
    if (ATOMIC_READ(prev->next) != ATOMIC_READ(next->node)) {

        prev = help_insert(prev, next);
        continue;
    }
```

Should release reference here

Should release reference here

Human fix

```
if (!prev) {
    release_ref(next->refcnt, ATOMIC_READ(next->node
        ));
    destroy_entry(entry);
    return -1;
}
while (1) {
    if (ATOMIC_READ(prev->next) != ATOMIC_READ(next
        ->node)) {
        release_ref(prev->refcnt, ATOMIC_READ(prev->
            node));
        prev = help_insert(prev, next);
        continue;
    }
```

Reference released

GPT-4o fix

```
if (!prev) {
    release_ref(next->refcnt, ATOMIC_READ(next->
        node));
    destroy_entry(entry);
    return -1;
}
while (1) {
    if (ATOMIC_READ(prev->next) != ATOMIC_READ(next
        ->node)) {

        prev = help_insert(prev, next);
        continue;
    }
```

Reference released

Missing reference release

Figure 6: GPT-4o fixed a bug partially.

Human fix

```
case 'r': str[j++] = '\x0d'; break;
case 't': str[j++] = '\x09'; break;
case 'v': str[j++] = '\x0b'; break;
case 'x':
case 'u':
case '\n':
case '\r':
default: str[j++] = '\\'; str[j++] = str[i];
```

Impl. details omitted

GPT-4o fix

```
case 'r': str[j++] = '\x0d'; break;
case 't': str[j++] = '\x09'; break;
case 'v': str[j++] = '\x0b'; break;
case 'x':
default: str[j++] = '\\'; str[j++] = str[i];
```

Impl. details omitted

Removed 3 switch cases: 'u', '\n', and '\r'

Figure 7: GPT-4o deleted functional code unrelated to the task.

We now discuss the two categories where the LLM-generated code was correct (top rows of Table 6).

Identical to Human Solution. For some commits, the LLM generates an exact replica of human-written code. While such behavior can be harmless when the reproduced code is correct, we observed that 5 out of 16 citations from GPT-4o across 212 commits cite the wrong resources[7] – using the buggy code prior to the fix – in the generated code, which left the bugs unfixed. This also confirms LLMs' vulnerability to data poisoning (generation of harmful code due to malicious data injected purposely into training data). For Ministral3 and Qwen3-Coder, the generated code does not contain any citations as they are local LLMs.

Different from Human, Appears Correct. Here the LLM generates the correct code for the intended fix or feature, though it differs from the human solution. For instance, when checking for integer overflow prior to calling `jsonp_malloc(len + 1)`, GPT-4o's code uses `if (len + 1 == 0)` whereas the human code uses the more robust version `if (len == (size_t) - 1)`.

Success Rate. The last three rows of Table 6 show the overall success rates for each project and LLM. The lowest success rate, 0%, is observed for packcc using Ministral3, and wolfSSL using Ministral3 and Qwen3-Coder. Overall, FFmpeg and wolfSSL have relatively low success rates as these projects have large code bases and large contexts (Table 1) where LLMs struggle to understand and produce a suitable, or even compilable, patch, as discussed next.

4.4 RQ4: What Determines the Success and Failure of LLM Code Generation?

We now discuss key factors contributing to LLM success or failure.

Context File Size and Function Size. As context file size and function size increase, the success rate first goes up and then drops, as shown in Figures 8 and 9. We believe the initial increase happens because very small files and functions may not give the LLM enough information to understand the task. With more context, the LLM gets traction – it can see more of the relevant behavior and dependencies. After a certain point, however, the file or function becomes too large for the LLM to understand reliably. The LLM

[7]For the commit shown in Figure 3, the LLM cited 2 GitHub repositories that use versions of *Collections-C* prior to the fix, alongside the generated code. These citations were provided as a file that contains code snippets with embedded URLs to their respective GitHub repositories. Anecdotally, this led to the LLM generating an empty patch, as the cited versions do not contain any fixes.

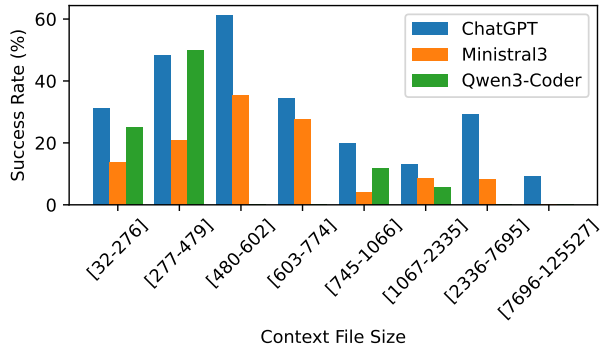


Figure 8: Success rate across file sizes (LOC).

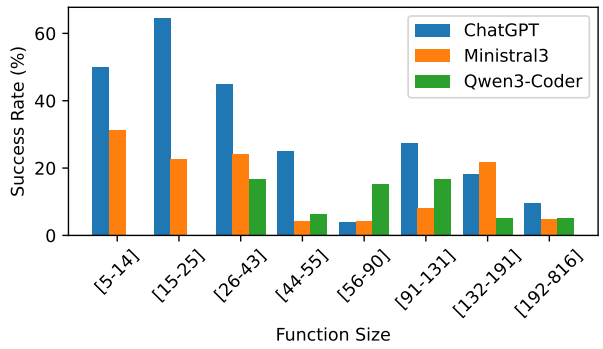


Figure 9: Success rate across function size (LOC).

Table 7: Statistical analysis: file size and function size.

Model	File Size (LoC)			Function Size (LoC)		
	Mean			Mean		
	Success	Failure	p-value	Success	Failure	p-value
GPT-4o	1316.6	4972.4	0.0053	50.4	105.7	0.0001
Ministral3	690.9	4339.6	0.0003	56.5	94.2	0.0180
Qwen3-Coder	551.3	8049.4	0.0005	101.8	151.2	0.0575

then has more code to track, the relevant changes become harder to isolate, and the success rate drops.

We used Welch's t-test [47] to validate these observed thresholds, by determining if the mean LoC differs significantly between successful and failed generations. As shown in Table 7, the results confirm that both file size and function size influence success. GPT-4o exhibited its strongest statistical separation at the function level ($p = 0.0001$): the mean successful function was 50.4 LoC compared to 105.7 LoC for failed functions, showing that larger function sizes pose significant challenge for its success. Across all models, p-values for file sizes range from 0.0003 to 0.0053, with failures often involving files several times larger than successes, e.g., Ministral3's mean failure at 4339.6 LoC vs. 690.1 LoC for success. For function size, p-values are significant (0.0001, 0.018) for GPT-4o as wel as Ministral 3, and indicative for Qwen3-Coder (0.0575). Therefore, the statistical results confirm that LLM's success is heavily dependent on code size.

Table 8: FFmpeg: success rate before and after the cutoff date.

		Success Rate	
		LOC<Median	LOC>Median
Knowledge Cutoff Date	Before	17.4% (4 out of 23)	13.6% (3 out of 22)
	After	14.3% (2 out of 14)	6.3% (1 out of 16)

Table 9: Success rate in different commits.

	Feature Enhancement	Bug Fix
Correct	13	56
Wrong	12	131
Success Rate	*52%*	*29.9%*

Pre- and Post-Knowledge Cutoff Dates. We performed a focused study on GPT-4o, which has a significantly larger parameter size than Ministral3 and Qwen3-Coder. We found that GPT-4o struggles to generate correct solutions for commits dated after GPT-4o's knowledge cutoff date (May 2024). We performed a focused analysis of FFmpeg, which has sufficient post-cutoff commit activity, and show the results in Table 8. For commits that precede the knowledge cutoff date the success rate was 13.6–17.4% depending on file size, whereas for commits that come after the knowledge cutoff date, the success rate dropped to 6.3–14.3%. This finding is concerning, because *LLMs will always operate de novo* when writing a patch: they could not have been trained on the very patch being written.

Nature of the Commit. For our second focused study, the LLM, GPT-4o, achieves a lower success rate (29.9%) for bug-fixing than feature-enhancements commits (Table 9). Although bug-fixing commits are smaller (median LOC=4), GPT-4o achieves a higher success rate (52%) on feature-enhancement commits (median LOC=15). We believe that locating and fixing a bug is harder for an LLM than implementing a new function because fixing bugs requires understanding the function, in addition to the context file. This aligns with our finding that success rate decreases with larger functions.

5 Discussion

Based on our findings, we provide several recommendations and discuss the limitations of our study.

5.1 Recommendations

Maintain a smaller context file size and function size. We observed that context file size and function size influence the LLMs' ability to generate correct code for commits. Keeping the context file under 603 LOC (albeit *not too small*) and function under 56 LOC increases the likelihood of correct code generation, as shown in Figures 8 and 9.

Stricter assessment on unfamiliar-task commits. The LLM struggles to generate correct code for commits that occur after the LLM's knowledge cutoff date (Table 8). This suggests that for unfamiliar or uncommon software engineering tasks, LLM-generated code should be subject to more rigorous human review and testing.

Bug-fixing commits require stronger scrutiny. Bug-fixing commits are more challenging for LLMs, as they require not only locating and fixing the bug, but also understanding both the context file and function. Due to their lower success rate (Table 9) we recommend stronger scrutiny of LLM-generated bug-fixing commits.

LLM selection for appropriate tasks. We found that the larger model, GPT-4o, generally performs better than smaller local LLMs, specifically in handling larger context (file/function size). Smaller context and function sizes are often assumed to be beneficial to smaller local LLMs, but in practice they do not reliably lead to correct code generation. The local LLMs perform well on routine and popular tasks, such as implementing data structures and algorithms, as shown in Table 6; as projects become more specialized, such as in *packcc* and *jansson*, they under-perform. Therefore, we suggest a larger model such as GPT-4o for patch generation, Qwen3-Coder (30B parameters in our setting) for exploratory or assistive coding, and Ministral3 (14B parameters) for resource-constrained settings.

5.2 Limitations

Generalizability. Our study is focused on a single language, C. This work could be extended to other languages to check for systematic, cross-language issues arising in LLM-based code generation. Our study examines commits affecting a single file and a single function. The study could be extended to a broader setting, commits affecting multiple files and functions. However, given the already-low success rate in the single file/function setting, we do not expect the success rate to improve when the setting is broader.

Commit size. The selected commits are limited to 66 LOC (Table 2) and contain changes to only a single file and a single function. We did not evaluate LLMs' capability in commits that involve changing multiple functions and files. However, our findings reveal that, even in our simplified setting, LLMs' ability to generate correct code deteriorates as the context (file or function) size increases.

Availability of commit messages as tasks (prompts). Our study relied on the availability of commit messages as task goals, i.e., prompts. When developing *de novo*, for bug fixes the prompts can come from bug summaries, after locating/reproducing bugs; for feature improvements, the prompts can come from feature requests.

Well-formed commit messages. Our study used well-formed commit messages (Section 3.3). Commits without well-formed messages (e.g., attaching a link to an issue rather than providing a detailed message, or lacking a rationale behind the change) drive the LLM to generate incorrect code. In such cases, the problem is the commit message, not the LLM; interestingly, LLMs can help automate the generation of high-quality commit messages [33, 44, 48].

6 Related Work

LLM Coding Evaluation. While Chong et al. [22] studied the security of LLM-generated code, we study LLMs' capabilities at fixing bugs and implementing new features. They analyzed early changes in Bison and Vsftpd's history; our examined changes are more recent, and do not overlap with the changes and period they studied. ObfusEval [53] uses obfuscated code to test LLMs' capability on unseen tasks; it revealed that the average pass rate of test suites for LLM-generated code can decrease by 15.3% to 62.5%, in line with our findings that the LLM struggles to generate code for commits made after its knowledge cutoff date. Sagodi et al. [39] studied automated Java vulnerability repair using manually engineered prompts. Other studies examined LLM-generated code's correctness and structural quality [18, 23, 24, 32], robustness [56], and performance regressions [30]. Wang et al. [45] categorized common types of LLM code generation errors using a benchmark dataset. Huang et al. [51] introduced ReinFix, an LLM-based program repair framework that guides patch generation using local and external code bases. FIXCHECK [36] assesses patch correctness using LLM-assisted test generation, while METAL [28] evaluates consistency of LLM outputs via metamorphic testing. Orthogonal to these efforts, we evaluate LLMs' capability in actual, essential software engineering tasks, rather than generic code generation.

LLM Hallucinations and Non-Determinism. LLMs can hallucinate, e.g., by generating syntactically correct but semantically flawed code [31, 54]. MARIN [21] aims to prevent LLMs from using non-existent APIs and misusing existing ones. Our Section 4.1 confirms such hallucinations. Additionally, the non-deterministic nature of LLMs means that the same prompts can yield different results, causing reproducibility issues [38]. Turbulence [27] shows that success on isolated code-generation prompts does not imply robustness, as LLMs frequently fail on closely related prompt instances that differ only in prompt-level constants.

Prompting with Better Context and Feedback Loop. LLM coding task benchmarks often fall short because they do not include the full or correct context (code base). HumanEvo [55] includes surrounding code and dependencies to more accurately assess generated code's quality. Zhang et al. [52] enhance LLMs' ability to modify code snippets by providing richer context and breaking down the tasks. To address the drawbacks of single-pass code generation, feedback loops have been introduced [22, 41, 43] to perform a cycle of testing and analysis on LLM generated code. Our framework also considers contextual references, but we do not perform a feedback loop because we evaluate the LLM system as-is.

7 Conclusions

Given the rise of LLM-generated code, we study three LLMs' viability for evolving real, open-source projects, by analyzing actual commits and combining verification&validation with manual inspection. LLMs are usable for small files and functions, feature improvements, and tasks similar to prior (seen) ones. In contrast, LLMs struggle with larger contexts, bug-fixing tasks, and new problems, often producing patches that fail silently. We outline recommendations to alleviate these issues. Our framework and findings can advance research in both enhancing LLMs' code generation capabilities, and verification&validation approaches for LLM-generated code.

Acknowledgments

We thank the reviewers for their valuable feedback. This material is based upon work supported by the National Science Foundation under Grant No. CCF-2106710 and an internal NJIT GHRI grant.

References

[1] [n. d.]. Clang: a C Language Family Frontend for LLVM. http://clang.llvm.org/.
[2] 2022. Research: quantifying GitHub Copilot's impact on developer productivity and happiness. https://github.blog/2022-09-07-research-quantifying-github-copilots-impact-on-developer-productivity-and-happiness.

[3] 2024. 2024 CWE Top 25 Most Dangerous Software Weaknesses. https://cwe.mitre.org/top25/archive/2024/2024_cwe_top25.html.

[4] 2024. GitHub Copilot - Your AI pair programmer. https://github.com/features/copilot.

[5] 2024. GitHub Copilot Enterprise on GPT-4o. https://github.blog/changelog/2024-07-05-github-copilot-enterprise-on-gpt-4o.

[6] 2024. Survey reveals AI's impact on the developer experience. https://github.blog/news-insights/research/survey-reveals-ais-impact-on-the-developer -experience.

[7] 2025. Bison. https://www.gnu.org/software/bison/.

[8] 2025. Collections-C. https://github.com/srdja/Collections-C.

[9] 2025. Cursor: The AI Code Editor. https://cursor.com/. Accessed: 2025-10-15.

[10] 2025. FFmpeg. https://github.com/FFmpeg/FFmpeg.

[11] 2025. IntelliCode. https://visualstudio.microsoft.com/services/intellicode/.

[12] 2025. jansson. https://github.com/akheron/jansson.

[13] 2025. libhl. https://github.com/xant/libhl.

[14] 2025. packcc. https://github.com/arithy/packcc.

[15] 2025. Prompt engineering for Copilot Chat. https://docs.github.com/en/copilot/concepts/prompt-engineering-for-copilot-chat.

[16] 2025. vsftpd. https://security.appspot.com/vsftpd.html.

[17] 2025. wolfSSL. https://github.com/wolfSSL/wolfssl.

[18] Victor Alves, Carla Bezerra, Ivan Machado, Larissa Rocha, Tássio Virgínio, and Publio Silva. 2025. Quality Assessment of Python Tests Generated by Large Language Models. *arXiv preprint arXiv:2506.14297* (2025).

[19] R.P.L. Buse and W. R. Weimer. 2010. Automatically documenting program changes. In *Proceedings of the 25th IEEE/ACM International Conference on Automated Software Engineering*. Association for Computing Machinery, 33–42.

[20] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. P. D. O. Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman, et al. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374* (2021).

[21] Y. Chen, M. Chen, C. Gao, Z. Jiang, Z. Li, and Y. Ma. 2025. Towards Mitigating API Hallucination in Code Generated by LLMs with Hierarchical Dependency Aware. In *Proceedings of the 33rd ACM International Conference on the Foundations of Software Engineering*. Association for Computing Machinery, 468–479.

[22] C. J. Chong, Z. Yao, and I. Neamtiu. 2024. Artificial-Intelligence Generated Code Considered Harmful: A Road Map for Secure and High-Quality Code Generation. *arXiv preprint arXiv:2409.19182* (2024).

[23] Tristan Coignion, Clément Quinton, and Romain Rouvoy. 2024. A performance study of llm-generated code on leetcode. In *Proceedings of the 28th international conference on evaluation and assessment in software engineering*. 79–89.

[24] Antonio Della Porta, Stefano Lambiase, and Fabio Palomba. 2025. Do Prompt Patterns Affect Code Quality? A First Empirical Assessment of ChatGPT-Generated Code. *arXiv preprint arXiv:2504.13656* (2025).

[25] Steve Easterbrook, Janice Singer, Margaret-Anne Storey, and Daniela Damian. 2008. *Selecting Empirical Methods for Software Engineering Research.* Springer London, London, 285–311. doi:10.1007/978-1-84800-044-5_11

[26] Israel Herraiz, Daniel Rodriguez, Gregorio Robles, and Jesus M. Gonzalez-Barahona. 2013. The evolution of the laws of software evolution: A discussion based on a systematic literature review. *ACM Comput. Surv.* 46, 2, Article 28 (Dec. 2013), 28 pages. doi:10.1145/2543581.2543595

[27] Shahin Honarvar, Mark van der Wilk, and Alastair F Donaldson. 2025. Turbulence: Systematically and automatically testing instruction-tuned large language models for code. In *2025 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 80–91.

[28] Sangwon Hyun, Mingyu Guo, and M Ali Babar. 2024. METAL: metamorphic testing framework for analyzing large-language model qualities. In *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 117–128.

[29] P. S. Kochhar, T. F. Bissyandé, D. Lo, and L. Jiang. 2013. An empirical study of adoption of software testing in open source projects. In *Proc. 13th International Conference on Quality Software*. IEEE, 103–112.

[30] S. Li, Y. Cheng, J. Chen, J. Xuan, S. He, and W. Shang. 2024. Assessing the Performance of AI-Generated Code: A Case Study on GitHub Copilot. In *2024 IEEE 35th ISSRE*. 216–227.

[31] F. Liu, Y. Liu, L. Shi, H. Huang, R. Wang, Z. Yang, L. Zhang, Z Li, and Y. Ma. 2024. Exploring and Evaluating Hallucinations in LLM-Powered Code Generation. *arXiv preprint arXiv:2404.00971* (2024).

[32] J. Liu, C. S. Xia, Y. Wang, and L. Zhang. 2023. Is your code generated by ChatGPT really correct? rigorous evaluation of large language models for code generation. In *Proc. NIPS*.

[33] S. Liu, C. Gao, S. Chen, L. Y. Nie, and Y. Liu. 2022. ATOM: Commit Message Generation Based on Abstract Syntax Tree and Hybrid Ranking. *IEEE Transactions on Software Engineering* 48, 5, 1800–1817.

[34] T. Liu, C. Xu, and J. McAuley. 2023. RepoBench: Benchmarking Repository-Level Code Auto-Completion Systems. *arXiv preprint arXiv:2306.03091* (2023).

[35] A. Mockus and L. G. Votta. 2000. Identifying reasons for software changes using historic databases. In *Proceedings 2000 International Conference on Software Maintenance*. 120–130.

[36] Facundo Molina, Juan Manuel Copia, and Alessandra Gorla. 2024. Improving Patch Correctness Analysis via Random Testing and Large Language Models. In *2024 IEEE Conference on Software Testing, Verification and Validation (ICST)*. IEEE, 317–328.

[37] Iulian Neamtiu, Guowu Xie, and Jianbo Chen. 2013. Towards a better understanding of software evolution: an empirical study on open-source software. *J. Softw. Evol. Process* 25, 3 (March 2013), 193–218. doi:10.1002/smr.564

[38] S Ouyang, J. M. Zhang, M. Harman, and M. Wang. 2025. An Empirical Study of the Non-Determinism of ChatGPT in Code Generation. *ACM Trans. Softw. Eng. Methodol.* 34, 2, Article 42 (2025).

[39] Zoltán Ságodi, Gábor Antal, Bence Bogenfürst, Martin Isztin, Péter Hegedűs, and Rudolf Ferenc. 2024. Reality check: Assessing GPT-4 in fixing real-world software vulnerabilities. In *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering*. 252–261.

[40] S. Shukla, H. Joshi, and R. Syed. 2025. Security Degradation in Iterative AI Code Generation – A Systematic Analysis of the Paradox. *arXiv preprint arXiv:2506.11022* (2025).

[41] H. Song, A. Goknil, X. Jiang, E. Melum, H. Joe, C. Gazzotti, V. Frascolla, A. N. Videsjorden, and P. Nguyen. 2025. Developing Multi-Agent LLM Applications Through Continuous Human-LLM Co-Programming. In *2025 IEEE/ACM 4th International Conference on AI Engineering – Software Engineering for AI (CAIN)*. IEEE Computer Society, 42–47.

[42] Y. Tian, Y. Zhang, K. Stol, L. Jiang, and H. Liu. 2022. What Makes a Good Commit Message?. In *2022 IEEE/ACM 44th ICSE*. 2389–2401.

[43] Z. Tian, J. Chen, and X. Zhang. 2025. Fixing Large Language Models' Specification Misunderstanding for Better Code Generation. In *2025 IEEE/ACM 47th ICSE*. IEEE Computer Society, 1514–1526.

[44] H. Wang, X. Xia, D. Lo, Q. He, X. Wang, and J. Grundy. 2021. Context-aware Retrieval-based Deep Commit Message Generation. *ACM Trans. Softw. Eng. Methodol.* 30, 4, Article 56 (2021).

[45] Z. Wang, Z. Zhou, D. Song, Y. Huang, S. Chen, L. Ma, and T. Zhang. 2025. Towards Understanding the Characteristics of Code Generation Errors Made by Large Language Models. In *2025 IEEE/ACM 47th ICSE*. IEEE Computer Society, 2587–2599.

[46] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou. 2022. Chain-of-thought prompting elicits reasoning in large language models. In *Proceedings of the 36th International Conference on Neural Information Processing Systems (NIPS '22)*. Curran Associates Inc., Article 1800, 14 pages.

[47] Bernard L Welch. 1947. The generalization of 'STUDENT'S'problem when several different population varlances are involved. *Biometrika* 34, 1-2 (1947), 28–35.

[48] Y. Wu, Y. Wang, Y. Li, W. Tao, S. Yu, H. Yang, W. Jiang, and J. Li. 2025. An Empirical Study on Commit Message Generation using LLMs via In-Context Learning. In *2025 IEEE/ACM 47th ICSE*. IEEE Computer Society, 553–565.

[49] H. Yu, B. Shen, D. Ran, J. Zhang, Q. Zhang, Y. Ma, G. Liang, Y. Li, Q. Wang, and T. Xie. 2024. CoderEval: A Benchmark of Pragmatic Code Generation with Generative Pre-trained Models. In *Proceedings of the IEEE/ACM 46th ICSE*. Association for Computing Machinery, Article 37.

[50] A. K. Zhang, N. Perry, R. Dulepet, E. Jones, J. W. Lin, J. Ji, C. Menders, G. Hussein, S. Liu, D. Jasper, et al. 2024. Cybench: A Framework for Evaluating Cybersecurity Capabilities and Risk of Language Models. *arXiv preprint arXiv:2408.08926* (2024).

[51] Jiayi Zhang, Kai Huang, Jian Zhang, Yang Liu, and Chunyang Chen. 2025. Repair Ingredients Are All You Need: Improving Large Language Model-Based Program Repair via Repair Ingredients Search. *arXiv preprint arXiv:2506.23100* (2025).

[52] T. Zhang, Y. Yu, X. Mao, S. Wang, K. Yang, Y. Lu, Z. Zhang, and Y. Zhao. 2025. Instruct or Interact? Exploring and Eliciting LLMs' Capability in Code Snippet Adaptation Through Prompt Engineering. In *2025 IEEE/ACM 47th ICSE*. IEEE Computer Society, 566–577.

[53] Y. Zhang, Y. Xie, S. Li, K. Liu, C. Wang, Z. Jia, X. Huang, J. Song, C. Luo, Z. Zheng, R. Xu, S. Liu, Y.and Zheng, and X. Liao. 2025. Unseen Horizons: Unveiling the Real Capability of LLM Code Generation Beyond the Familiar. In *2025 IEEE/ACM 47th ICSE*. IEEE Computer Society, 604–615.

[54] Z. Zhang, C. Wang, Y. Wang, E. Shi, Y. Ma, W. Zhong, J. Chen, M. Mao, and Z. Zheng. 2025. Llm hallucinations in practical code generation: Phenomena, mechanism, and mitigation. *Proceedings of the ACM on Software Engineering* 2, ISSTA (2025), 481–503.

[55] D. Zheng, Y. Wang, E. Shi, R. Zhang, Y. Ma, H. Zhang, and Z. Zheng. 2025. HumanEvo: An Evolution-aware Benchmark for More Realistic Evaluation of Repository-level Code Generation. In *2025 IEEE/ACM 47th ICSE*. IEEE Computer Society, 764–764.

[56] L. Zhong and Z. Wang. 2024. Can llm replace stack overflow? a study on robustness and reliability of large language model code generation. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 38. 21841–21849.